Time Matters Java program

src/demo/Appointment.java

```
 1  package demo;
 2
 3  import java.time.LocalDate;
 4  import java.time.LocalTime;
 5  import java.time.format.DateTimeFormatter;
 6  import java.time.format.DateTimeParseException;
 7  import java.time.format.ResolverStyle;
 8  import java.time.temporal.ChronoUnit;
 9  import java.util.Objects;
10
11  public final class Appointment {
12      public enum Repeat { NONE, DAILY, WEEKLY }
13      private static final DateTimeFormatter DATE_FORMAT = DateTimeFormatter
14              .ofPattern("uuuu-MM-dd").withResolverStyle(ResolverStyle.STRICT);
15      private static final DateTimeFormatter TIME_FORMAT = DateTimeFormatter
16              .ofPattern("HH:mm").withResolverStyle(ResolverStyle.STRICT);
17
18      private final String title;
19      private final LocalDate date;
20      private final LocalTime start;
21      private final LocalTime end;
22      private final String location;
23      private final Repeat repeat;
24      private final LocalDate repeatUntil;
25      private final int reminderMinutes;
26
27      public Appointment(String title, LocalDate date, LocalTime start, LocalTime end,
28              String location) {
29          this(title, date, start, end, location, Repeat.NONE, null, 0);
30      }
31
32      public Appointment(String title, LocalDate date, LocalTime start, LocalTime end,
33              String location, Repeat repeat, LocalDate repeatUntil, int reminderMinutes) {
34          if (title == null || title.isBlank()) {
35              throw new IllegalArgumentException("Enter an appointment title.");
36          }
37          if (title.trim().length() > 60) {
38              throw new IllegalArgumentException("Use no more than 60 characters for the title.");
39          }
40          if (date == null || start == null || end == null) {
41              throw new IllegalArgumentException("Date, start time and end time are required.");
42          }
43          if (!end.isAfter(start)) {
```

Time Matters Java program

src/demo/Appointment.java continued

```
44              throw new IllegalArgumentException("End time must be after start time on the same day.");
45          }
46          this.title = title.trim();
47          this.date = date;
48          this.start = start;
49          this.end = end;
50          this.location = location == null ? "" : location.trim();
51          this.repeat = Objects.requireNonNull(repeat);
52          if (repeat != Repeat.NONE && (repeatUntil == null || repeatUntil.isBefore(date)
53                  || ChronoUnit.DAYS.between(date, repeatUntil) > 366)) {
54              throw new IllegalArgumentException("Repeat until must be on or within 366 days of the start date.");
55          }
56          if (reminderMinutes < 0 || reminderMinutes > 1440) {
57              throw new IllegalArgumentException("Reminder must be between 0 and 1440 minutes.");
58          }
59          this.repeatUntil = repeat == Repeat.NONE ? date : repeatUntil;
60          this.reminderMinutes = reminderMinutes;
61      }
62
63      public static Appointment fromInput(String title, String date, String start,
64              String end, String location) {
65          LocalDate parsedDate;
66          LocalTime parsedStart;
67          LocalTime parsedEnd;
68          try {
69              parsedDate = LocalDate.parse(clean(date), DATE_FORMAT);
70          } catch (DateTimeParseException ex) {
71              throw new IllegalArgumentException("Enter a valid date as YYYY-MM-DD.");
72          }
73          try {
74              parsedStart = LocalTime.parse(clean(start), TIME_FORMAT);
75              parsedEnd = LocalTime.parse(clean(end), TIME_FORMAT);
76          } catch (DateTimeParseException ex) {
77              throw new IllegalArgumentException("Enter valid times as HH:mm, for example 09:00.");
78          }
79          return new Appointment(title, parsedDate, parsedStart, parsedEnd, location);
80      }
81
82      private static String clean(String value) { return value == null ? "" : value.trim(); }
83
84      public String getTitle() { return title; }
85      public LocalDate getDate() { return date; }
86      public LocalTime getStart() { return start; }
```

Time Matters Java program

src/demo/Appointment.java continued

```
 87      public LocalTime getEnd() { return end; }
 88      public String getLocation() { return location; }
 89      public Repeat getRepeat() { return repeat; }
 90      public LocalDate getRepeatUntil() { return repeatUntil; }
 91      public int getReminderMinutes() { return reminderMinutes; }
 92
 93      public boolean occursOn(LocalDate day) {
 94          if (day.isBefore(date) || day.isAfter(repeatUntil)) return false;
 95          return repeat != Repeat.WEEKLY || day.getDayOfWeek() == date.getDayOfWeek();
 96      }
 97
 98      public Object[] toTableRow() {
 99          return new Object[]{title, date.toString(), start.format(TIME_FORMAT),
100              end.format(TIME_FORMAT), location};
101      }
102  }
```

Time Matters Java program

src/demo/AppointmentBook.java

```
 1  package demo;
 2
 3  import java.io.IOException;
 4  import java.nio.charset.StandardCharsets;
 5  import java.nio.file.*;
 6  import java.time.*;
 7  import java.time.temporal.ChronoUnit;
 8  import java.util.*;
 9  import java.util.stream.Stream;
10
11  public final class AppointmentBook {
12      private final Path file;
13      private byte[] savedBytes;
14      private List<Appointment> entries = new ArrayList<>();
15
16      public AppointmentBook(Path file) throws IOException {
17          this.file = file.toAbsolutePath();
18          if (Files.exists(this.file)) {
19              savedBytes = Files.readAllBytes(this.file);
20              List<String> lines = new String(savedBytes, StandardCharsets.UTF_8).lines().toList();
21              if (lines.isEmpty() || !Set.of("SCHEDULER-3", "SCHEDULER-4").contains(lines.get(0))) {
22                  throw new IOException("Unrecognised appointment file: " + this.file);
23              }
24              try {
25                  boolean legacy = lines.get(0).equals("SCHEDULER-3");
26                  for (int i = 1; i < lines.size(); i++) {
27                      String[] row = lines.get(i).split("\t", -1);
28                      if (row.length != (legacy ? 5 : 8)) throw new IllegalArgumentException("Invalid row");
29                      entries.add(new Appointment(decode(row[0]), LocalDate.parse(row[1]),
30                              LocalTime.parse(row[2]), LocalTime.parse(row[3]), decode(row[4]),
31                              legacy ? Appointment.Repeat.NONE : Appointment.Repeat.valueOf(row[5]),
32                              legacy ? null : LocalDate.parse(row[6]), legacy ? 0 : Integer.parseInt(row[7])));
33                  }
34              } catch (RuntimeException ex) {
35                  throw new IOException("Appointment file contains invalid data: " + this.file, ex);
36              }
37          }
38      }
39
40      public List<Appointment> getAll() { return List.copyOf(entries); }
41
42      public void add(Appointment appointment) throws IOException {
43          Objects.requireNonNull(appointment);
```

Time Matters Java program

src/demo/AppointmentBook.java continued

```
44          checkConflicts(appointment, -1);
45          List<Appointment> next = new ArrayList<>(entries);
46          next.add(appointment);
47          save(next);
48      }
49
50      public void update(int index, Appointment appointment) throws IOException {
51          Objects.requireNonNull(appointment);
52          List<Appointment> next = new ArrayList<>(entries);
53          next.set(index, appointment);
54          checkConflicts(appointment, index);
55          save(next);
56      }
57
58      public void delete(int index) throws IOException {
59          List<Appointment> next = new ArrayList<>(entries);
60          next.remove(index);
61          save(next);
62      }
63
64      public List<Occurrence> view(LocalDate anchor, String period) {
65          LocalDate from = switch (period) {
66              case "Week" -> anchor.with(DayOfWeek.MONDAY);
67              case "Month" -> anchor.withDayOfMonth(1);
68              default -> anchor;
69          };
70          LocalDate to = switch (period) {
71              case "Week" -> from.plusDays(6);
72              case "Month" -> from.withDayOfMonth(from.lengthOfMonth());
73              default -> from;
74          };
75          List<Occurrence> matches = new ArrayList<>();
76          for (Appointment appointment : entries) {
77              dates(from, to).filter(appointment::occursOn)
78                      .forEach(date -> matches.add(new Occurrence(appointment, date)));
79          }
80          matches.sort(Comparator.comparing(Occurrence::start));
81          return matches;
82      }
83
84      private void checkConflicts(Appointment candidate, int excludedIndex) {
85          for (int i = 0; i < entries.size(); i++) {
86              Appointment existing = entries.get(i);
```

Time Matters Java program

src/demo/AppointmentBook.java continued

```
            if (i == excludedIndex || !candidate.getStart().isBefore(existing.getEnd())
                    || !existing.getStart().isBefore(candidate.getEnd())) continue;
            Optional<LocalDate> clash = dates(candidate.getDate(), candidate.getRepeatUntil())
                    .filter(candidate::occursOn).filter(existing::occursOn).findFirst();
            if (clash.isPresent()) {
                throw new IllegalArgumentException("Overlaps " + existing.getTitle() + " on " + clash.get() + ".");
            }
        }
    }

    private static Stream<LocalDate> dates(LocalDate from, LocalDate to) {
        return Stream.iterate(from, day -> day.plusDays(1))
                .limit(ChronoUnit.DAYS.between(from, to) + 1);
    }

    private static String encode(String value) {
        return Base64.getEncoder().encodeToString(value.getBytes(StandardCharsets.UTF_8));
    }

    private static String decode(String value) {
        return new String(Base64.getDecoder().decode(value), StandardCharsets.UTF_8);
    }

    /** Replace the data file only after the complete new file has been written. */
    private void save(List<Appointment> next) throws IOException {
        byte[] current = Files.exists(file) ? Files.readAllBytes(file) : null;
        if (!Arrays.equals(savedBytes, current)) {
            throw new IOException("Data changed in another window. Close and reopen this window.");
        }
        Path temporary = Files.createTempFile(file.getParent(), "appointments-", ".tmp");
        try {
            List<String> lines = new ArrayList<>();
            lines.add("SCHEDULER-4");
            for (Appointment a : next) {
                lines.add(String.join("\t", encode(a.getTitle()), a.getDate().toString(),
                        a.getStart().toString(), a.getEnd().toString(), encode(a.getLocation()),
                        a.getRepeat().name(), a.getRepeatUntil().toString(),
                        Integer.toString(a.getReminderMinutes())));
            }
            Files.write(temporary, lines, StandardCharsets.UTF_8);
            try {
                Files.move(temporary, file, StandardCopyOption.ATOMIC_MOVE,
                        StandardCopyOption.REPLACE_EXISTING);
```

Time Matters Java program

src/demo/AppointmentBook.java continued

```
130             } catch (AtomicMoveNotSupportedException ex) {
131                 Files.move(temporary, file, StandardCopyOption.REPLACE_EXISTING);
132             }
133             entries = next;
134             savedBytes = Files.readAllBytes(file);
135         } finally {
136             Files.deleteIfExists(temporary);
137         }
138     }
139 }
```

Time Matters Java program

src/demo/AppTheme.java

```
 1  package demo;
 2
 3  import java.awt.*;
 4  import javax.swing.*;
 5  import javax.swing.table.DefaultTableCellRenderer;
 6  import javax.swing.text.JTextComponent;
 7
 8  final class AppTheme {
 9      static final Color BACKGROUND = new Color(242, 248, 250);
10      static final Color SURFACE = Color.WHITE;
11      static final Color CYAN = new Color(0, 204, 255);
12      static final Color TEXT = new Color(25, 43, 52);
13      static final Color MUTED = new Color(99, 116, 126);
14      static final Color LINE = new Color(183, 204, 214);
15      static final Color INK = new Color(0, 92, 116);
16
17      static void install() {
18          UIManager.put("control", SURFACE);
19          UIManager.put("text", SURFACE);
20          UIManager.put("controlText", TEXT);
21          UIManager.put("textText", TEXT);
22          UIManager.put("textHighlight", CYAN);
23          UIManager.put("textHighlightText", Color.BLACK);
24          UIManager.put("Button.disabledText", MUTED);
25          UIManager.put("ComboBox.disabledForeground", MUTED);
26          UIManager.put("ComboBox.disabledBackground", SURFACE);
27      }
28
29      static void style(Component component) {
30          component.setFont(new Font("Segoe UI", Font.PLAIN, 13));
31          component.setForeground(TEXT);
32          if (component instanceof JPanel || component instanceof JViewport) component.setBackground(BACKGROUND);
33          if (component instanceof JLabel label) {
34              label.setForeground(TEXT);
35              label.setBorder(BorderFactory.createEmptyBorder(0, 0, 0, 5));
36          }
37          if (component instanceof JTextComponent input) {
38              input.setBackground(SURFACE);
39              input.setCaretColor(INK);
40              input.setSelectionColor(CYAN);
41              input.setSelectedTextColor(Color.BLACK);
42              input.setDisabledTextColor(MUTED);
43              input.setBorder(BorderFactory.createCompoundBorder(
```

Time Matters Java program

src/demo/AppTheme.java continued

```
                    BorderFactory.createLineBorder(LINE), BorderFactory.createEmptyBorder(7, 8, 7, 8)));
        }
        if (component instanceof JButton button) {
            button.setBackground(SURFACE);
            button.setForeground(INK);
            button.setFocusPainted(false);
            button.setMargin(new Insets(8, 12, 8, 12));
        }
        if (component instanceof JComboBox<?> box) box.setBackground(SURFACE);
        if (component instanceof JSeparator separator) separator.setForeground(LINE);
        if (component instanceof JScrollPane scroll) scroll.setBorder(BorderFactory.createLineBorder(LINE));
        if (component instanceof Container container)
            for (Component child : container.getComponents()) style(child);
    }

    static void welcome(Container container) {
        container.setBackground(Color.BLACK);
        for (Component child : container.getComponents()) {
            if (child instanceof JLabel) child.setForeground(CYAN);
            if (child instanceof Container nested) welcome(nested);
        }
    }

    static void primary(JButton button) {
        button.setBackground(CYAN);
        button.setForeground(Color.BLACK);
        button.setFont(new Font("Segoe UI", Font.BOLD, 14));
    }

    static void table(JTable table) {
        table.setBackground(SURFACE);
        table.setForeground(TEXT);
        table.setSelectionBackground(CYAN);
        table.setSelectionForeground(Color.BLACK);
        table.setRowHeight(34);
        table.setShowGrid(false);
        table.getTableHeader().setDefaultRenderer(new DefaultTableCellRenderer() {
            @Override public Component getTableCellRendererComponent(JTable t, Object value,
                    boolean selected, boolean focused, int row, int column) {
                super.getTableCellRendererComponent(t, value, selected, focused, row, column);
                setBackground(new Color(20, 26, 31));
                setForeground(CYAN);
                setFont(new Font("Segoe UI", Font.BOLD, 13));
```

Time Matters Java program

src/demo/AppTheme.java continued

```
 87                 setBorder(BorderFactory.createEmptyBorder(9, 8, 9, 8));
 88                 return this;
 89             }
 90         });
 91         table.getTableHeader().setPreferredSize(new Dimension(0, 38));
 92         table.setDefaultRenderer(Object.class, new DefaultTableCellRenderer() {
 93             @Override public Component getTableCellRendererComponent(JTable t, Object value,
 94                     boolean selected, boolean focused, int row, int column) {
 95                 super.getTableCellRendererComponent(t, value, selected, focused, row, column);
 96                 setBackground(selected ? CYAN : row % 2 == 0 ? SURFACE : new Color(233, 245, 249));
 97                 setForeground(selected ? Color.BLACK : TEXT);
 98                 setBorder(BorderFactory.createEmptyBorder(5, 8, 5, 8));
 99                 return this;
100             }
101         });
102     }
103 }
```

Time Matters Java program

src/demo/CalendarExport.java

```
 1  package demo;
 2
 3  import java.io.IOException;
 4  import java.nio.charset.StandardCharsets;
 5  import java.nio.file.Files;
 6  import java.nio.file.Path;
 7  import java.nio.file.StandardCopyOption;
 8  import java.time.DateTimeException;
 9  import java.time.Instant;
10  import java.time.LocalDateTime;
11  import java.time.ZoneId;
12  import java.time.ZoneOffset;
13  import java.time.format.DateTimeFormatter;
14  import java.util.List;
15  import java.util.Locale;
16  import java.util.UUID;
17
18  public final class CalendarExport {
19      private static final DateTimeFormatter UTC = DateTimeFormatter
20              .ofPattern("uuuuMMdd'T'HHmmss'Z'", Locale.ROOT).withZone(ZoneOffset.UTC);
21
22      private CalendarExport() {}
23
24      public static void write(Path target, List<Occurrence> rows, ZoneId zone, boolean replace)
25              throws IOException {
26          String calendar = render(rows, zone, Instant.now());
27          Path destination = target.toAbsolutePath();
28          Path temporary = Files.createTempFile(destination.getParent(), "calendar-", ".tmp");
29          try {
30              Files.writeString(temporary, calendar, StandardCharsets.UTF_8);
31              if (replace) Files.move(temporary, destination, StandardCopyOption.REPLACE_EXISTING);
32              else Files.move(temporary, destination);
33          } finally {
34              Files.deleteIfExists(temporary);
35          }
36      }
37
38      static String render(List<Occurrence> rows, ZoneId zone, Instant timestamp) {
39          StringBuilder output = new StringBuilder();
40          line(output, "BEGIN:VCALENDAR");
41          line(output, "VERSION:2.0");
42          line(output, "PRODID:-//SDN150S Group 12//Appointment Scheduler//EN");
43          for (Occurrence row : rows) {
```

Time Matters Java program

src/demo/CalendarExport.java continued

```
            Appointment appointment = row.appointment();
            line(output, "BEGIN:VEVENT");
            line(output, "UID:" + UUID.randomUUID() + "@appointment-scheduler.local");
            line(output, "DTSTAMP:" + UTC.format(timestamp));
            line(output, "DTSTART:" + eventTime(row.start(), zone));
            line(output, "DTEND:" + eventTime(row.date().atTime(appointment.getEnd()), zone));
            line(output, "SUMMARY:" + escape(appointment.getTitle()));
            if (!appointment.getLocation().isEmpty()) {
                line(output, "LOCATION:" + escape(appointment.getLocation()));
            }
            line(output, "END:VEVENT");
        }
        line(output, "END:VCALENDAR");
        return output.toString();
    }

    private static String eventTime(LocalDateTime time, ZoneId zone) {
        List<ZoneOffset> offsets = zone.getRules().getValidOffsets(time);
        if (offsets.isEmpty()) {
            throw new DateTimeException(time + " does not exist in " + zone + " due to a clock change.");
        }
        Instant instant = time.toInstant(offsets.getFirst());
        int year = instant.atOffset(ZoneOffset.UTC).getYear();
        if (year < 1 || year > 9999) throw new DateTimeException("Calendar export requires years 0001 to 9999.");
        return UTC.format(instant);
    }

    private static String escape(String value) {
        if (value.codePoints().anyMatch(c -> c < 32 && c != '\t' && c != '\r' && c != '\n' || c == 127)) {
            throw new IllegalArgumentException("An appointment contains an unsupported control character.");
        }
        return value.replace("\\", "\\\\").replace(",", "\\,").replace(";", "\\;")
                .replace("\r\n", "\\n").replace("\n", "\\n").replace("\r", "\\n");
    }

    /** Fold by UTF-8 bytes without splitting a Unicode character. */
    private static void line(StringBuilder output, String value) {
        int bytes = 0;
        for (int index = 0; index < value.length();) {
            int codePoint = value.codePointAt(index);
            String character = Character.toString(codePoint);
            int size = character.getBytes(StandardCharsets.UTF_8).length;
            if (bytes + size > 75) {
```

Time Matters Java program

src/demo/CalendarExport.java continued

```
87                  output.append("\r\n ");
88                  bytes = 1;
89              }
90              output.append(character);
91              bytes += size;
92              index += Character.charCount(codePoint);
93          }
94          output.append("\r\n");
95      }
96  }
```

Time Matters Java program

src/demo/Main.java

```
 1  package demo;
 2
 3  import javax.swing.SwingUtilities;
 4  import javax.swing.UIManager;
 5
 6  /**
 7   * @author Noluthando Basi (GitHub: ZuluBarbie)
 8   */
 9  public class Main {
10      public static void main(String[] args) {
11          SwingUtilities.invokeLater(() -> {
12              try {
13                  UIManager.setLookAndFeel(UIManager.getCrossPlatformLookAndFeelClassName());
14              } catch (Exception ex) {
15                  System.err.println("Using default appearance: " + ex.getMessage());
16              }
17              AppTheme.install();
18              new NewJFrame1().setVisible(true);
19          });
20      }
21  }
```

Time Matters Java program

src/demo/NewJFrame1.java

```
 1  package demo;
 2
 3  import java.awt.*;
 4  import javax.swing.*;
 5  import java.io.IOException;
 6  import java.nio.file.Files;
 7  import java.nio.file.Path;
 8
 9  public class NewJFrame1 extends JFrame {
10      public NewJFrame1() {
11          setTitle("Time Matters - Appointment Scheduler");
12          setDefaultCloseOperation(WindowConstants.DISPOSE_ON_CLOSE);
13          JPanel page = new JPanel(new GridBagLayout());
14          page.setBorder(BorderFactory.createEmptyBorder(36, 36, 36, 36));
15          JPanel content = new JPanel(new GridBagLayout());
16          GridBagConstraints row = new GridBagConstraints();
17          row.gridx = 0;
18          row.fill = GridBagConstraints.HORIZONTAL;
19          row.insets = new Insets(8, 0, 8, 0);
20          JLabel title = new JLabel("Time Matters", SwingConstants.CENTER);
21          JLabel tagline = new JLabel("YOUR TIME. YOUR SCHEDULE", SwingConstants.CENTER);
22          JSeparator divider = new JSeparator();
23          JButton open = new JButton("Open scheduler");
24          JButton documents = new JButton("Submission files");
25          documents.addActionListener(e -> openDocuments());
26          JLabel caption = new JLabel("Plan your day with Time Matters", SwingConstants.CENTER);
27          JComponent[] items = {title, tagline, divider, open, documents, caption};
28          for (int i = 0; i < items.length; i++) {
29              row.gridy = i;
30              content.add(items[i], row);
31          }
32          page.add(content);
33          setContentPane(page);
34          AppTheme.style(page);
35          AppTheme.welcome(page);
36          title.setFont(new Font("Segoe UI", Font.BOLD, 38));
37          tagline.setFont(new Font("Segoe UI", Font.PLAIN, 14));
38          caption.setFont(new Font("Segoe UI", Font.ITALIC, 12));
39          divider.setPreferredSize(new Dimension(320, 1));
40          open.setPreferredSize(new Dimension(320, 42));
41          AppTheme.primary(open);
42          documents.setPreferredSize(new Dimension(320, 36));
43          documents.setBackground(new Color(20, 26, 31));
```

Time Matters Java program

src/demo/NewJFrame1.java continued

```
44          documents.setForeground(AppTheme.CYAN);
45          open.addActionListener(e -> {
46              open.setEnabled(false);
47              try {
48                  new SchedulerFrame().setVisible(true);
49                  dispose();
50              } catch (RuntimeException ex) {
51                  open.setEnabled(true);
52                  JOptionPane.showMessageDialog(this, "Could not open scheduler: " + ex.getMessage(),
53                          "Time Matters", JOptionPane.ERROR_MESSAGE);
54              }
55          });
56          getRootPane().setDefaultButton(open);
57          setMinimumSize(new Dimension(500, 360));
58          setSize(640, 480);
59          setLocationRelativeTo(null);
60      }
61
62      static Path submissionDirectory() throws IOException {
63          Path location = Path.of(System.getProperty("user.dir")).toAbsolutePath();
64          while (location != null) {
65              Path folder = location.resolve("Submission");
66              if (Files.isDirectory(folder)) return folder;
67              location = location.getParent();
68          }
69          throw new IOException("Submission folder not found. Open the app from its project folder.");
70      }
71
72      private void openDocuments() {
73          try {
74              if (!Desktop.isDesktopSupported() || !Desktop.getDesktop().isSupported(Desktop.Action.OPEN)) {
75                  throw new IOException("Open the Submission folder in File Explorer.");
76              }
77              Desktop.getDesktop().open(submissionDirectory().toFile());
78          } catch (IOException | SecurityException ex) {
79              JOptionPane.showMessageDialog(this, ex.getMessage(), "Submission files", JOptionPane.INFORMATION_MESSAGE);
80          }
81      }
82
83      public static void main(String[] args) { Main.main(args); }
84  }
```

Time Matters Java program

src/demo/Occurrence.java

```
 1  package demo;
 2
 3  import java.time.LocalDate;
 4  import java.time.LocalDateTime;
 5
 6  public record Occurrence(Appointment appointment, LocalDate date) {
 7      public LocalDateTime start() { return date.atTime(appointment.getStart()); }
 8
 9      public Object[] toTableRow() {
10          Object[] base = appointment.toTableRow();
11          return new Object[]{base[0], date, base[2], base[3], base[4],
12              appointment.getRepeat(), appointment.getReminderMinutes()};
13      }
14  }
```

Time Matters Java program

src/demo/Reminders.java

```
package demo;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

public final class Reminders {
    private final Set<Occurrence> shown = new HashSet<>();

    public List<Occurrence> due(List<Appointment> appointments, LocalDateTime now) {
        shown.removeIf(o -> !o.start().isAfter(now) || !appointments.contains(o.appointment()));
        List<Occurrence> due = new ArrayList<>();
        LocalDate today = now.toLocalDate();
        for (Appointment appointment : appointments) {
            int minutes = appointment.getReminderMinutes();
            if (minutes == 0) continue;
            for (int offset = 0; offset <= 1; offset++) {
                LocalDate date = today.plusDays(offset);
                if (!appointment.occursOn(date)) continue;
                Occurrence occurrence = new Occurrence(appointment, date);
                LocalDateTime start = occurrence.start();
                if (start.isAfter(now) && !now.isBefore(start.minusMinutes(minutes))
                        && shown.add(occurrence)) {
                    due.add(occurrence);
                }
            }
        }
        return due;
    }
}
```

Time Matters Java program

src/demo/SchedulerFrame.java

```
 1  package demo;
 2
 3  import java.awt.Color;
 4  import java.io.IOException;
 5  import java.nio.file.Files;
 6  import java.nio.file.Path;
 7  import java.time.LocalDate;
 8  import java.time.LocalDateTime;
 9  import java.time.DateTimeException;
10  import java.time.ZoneId;
11  import java.util.List;
12  import java.util.Locale;
13  import javax.swing.*;
14  import javax.swing.table.DefaultTableModel;
15
16  public class SchedulerFrame extends JFrame {
17      private AppointmentBook book;
18      private LocalDate anchor = LocalDate.now();
19      private List<Occurrence> visible = List.of();
20      private final JComboBox<Appointment.Repeat> repeatBox = new JComboBox<>(Appointment.Repeat.values());
21      private final JTextField untilField = new JTextField(9);
22      private final JSpinner reminderField = new JSpinner(new SpinnerNumberModel(0, 0, 1440, 1));
23      private final Reminders reminders = new Reminders();
24      private final DefaultTableModel model = new DefaultTableModel(
25              new String[]{"Title", "Date", "Start", "End", "Location", "Repeat", "Reminder (min)"}, 0) {
26          @Override public boolean isCellEditable(int row, int column) { return false; }
27      };
28
29      @SuppressWarnings("unchecked")
30      public SchedulerFrame() {
31          initComponents();
32          JPanel options = new JPanel(new java.awt.FlowLayout(java.awt.FlowLayout.LEFT));
33          addOption(options, "Repeat", repeatBox);
34          addOption(options, "Repeat until (YYYY-MM-DD)", untilField);
35          addOption(options, "Reminder minutes (0 = off)", reminderField);
36          untilField.setEnabled(false);
37          repeatBox.addActionListener(e -> untilField.setEnabled(repeatBox.getSelectedItem() != Appointment.Repeat.NONE));
38          JPanel footer = new JPanel(new java.awt.BorderLayout());
39          footer.add(options, java.awt.BorderLayout.NORTH);
40          footer.add(statusLabel, java.awt.BorderLayout.SOUTH);
41          getContentPane().add(footer, java.awt.BorderLayout.SOUTH);
42          ((JPanel) getContentPane()).setBorder(BorderFactory.createEmptyBorder(16,16,16,16));
43          periodBox.setModel(new DefaultComboBoxModel<>(new String[]{"Day", "Week", "Month"}));
```

Time Matters Java program

src/demo/SchedulerFrame.java continued

```
        anchorField.setColumns(10);
        table.setModel(model);
        table.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
        table.setFillsViewportHeight(true);
        table.setRowHeight(28);
        String[] labels = {"Title", "Appointment date", "Start time", "End time", "Location", "View date"};
        JTextField[] inputs = {titleField,dateField,startField,endField,locationField,anchorField};
        for (int i=0;i<inputs.length;i++) { inputs[i].getAccessibleContext().setAccessibleName(labels[i]); }
        table.getAccessibleContext().setAccessibleName("Appointments");
        addButton.addActionListener(e -> save(false));
        updateButton.addActionListener(e -> save(true));
        deleteButton.addActionListener(e -> deleteSelected());
        clearButton.addActionListener(e -> clear());
        exportButton.addActionListener(e -> exportView());
        table.getSelectionModel().addListSelectionListener(e -> { if (!e.getValueIsAdjusting()) select(); });
        showButton.addActionListener(e -> showDate());
        anchorField.addActionListener(e -> showDate());
        periodBox.addActionListener(e -> refresh());
        previousButton.addActionListener(e -> move(-1));
        nextButton.addActionListener(e -> move(1));
        todayButton.addActionListener(e -> { anchor=LocalDate.now(); refresh(); });
        try {
            book = new AppointmentBook(Path.of(System.getProperty("scheduler.dataFile","appointments.tsv")));
        } catch (IOException ex) {
            addButton.setEnabled(false);
            message("Cannot load data. File preserved: " + ex.getMessage(),true);
            JOptionPane.showMessageDialog(this,ex.getMessage(),"Could not load appointments",JOptionPane.ERROR_MESSAGE);
        }
        clear();
        refresh();
        if (book != null) {
            message("Loaded " + book.getAll().size() + " appointments. Keep this window open for reminders.", false);
            Timer timer = new Timer(1000, e -> checkReminders());
            addWindowListener(new java.awt.event.WindowAdapter() {
                @Override public void windowClosed(java.awt.event.WindowEvent event) { timer.stop(); }
            });
            timer.start();
        }
        AppTheme.style(getContentPane());
        AppTheme.primary(addButton);
        AppTheme.table(table);
        setTitle("Time Matters - Appointment Scheduler");
        options.setBorder(BorderFactory.createEmptyBorder(12, 0, 12, 0));
```

Time Matters Java program

src/demo/SchedulerFrame.java continued

```
 87          statusLabel.setBorder(BorderFactory.createEmptyBorder(6, 0, 0, 0));
 88          statusLabel.setForeground(AppTheme.INK);
 89          setMinimumSize(new java.awt.Dimension(1120,560));
 90          setSize(1240,680);
 91          setLocationRelativeTo(null);
 92      }
 93
 94      private static void addOption(JPanel panel, String text, JComponent input) {
 95          JLabel label = new JLabel(text);
 96          label.setLabelFor(input);
 97          panel.add(label);
 98          panel.add(input);
 99      }
100
101      private void checkReminders() {
102          List<Occurrence> due = reminders.due(book.getAll(), LocalDateTime.now());
103          if (due.isEmpty()) return;
104          StringBuilder message = new StringBuilder();
105          for (Occurrence occurrence : due) {
106              message.append(occurrence.appointment().getTitle()).append('\n').append("Starts ")
107                      .append(occurrence.date()).append(" at ").append(occurrence.appointment().getStart())
108                      .append("\n\n");
109          }
110          JTextArea details = new JTextArea(message.toString().stripTrailing(), Math.min(8, due.size() * 3), 34);
111          details.setEditable(false);
112          details.setLineWrap(true);
113          details.setWrapStyleWord(true);
114          details.setCaretPosition(0);
115          JOptionPane pane = new JOptionPane(new JScrollPane(details), JOptionPane.INFORMATION_MESSAGE);
116          JDialog dialog = pane.createDialog(this, "Appointment reminder");
117          dialog.setModal(false);
118          dialog.setDefaultCloseOperation(WindowConstants.DISPOSE_ON_CLOSE);
119          pane.addPropertyChangeListener(JOptionPane.VALUE_PROPERTY, e -> dialog.dispose());
120          dialog.setVisible(true);
121      }
122
123      private void message(String text, boolean error) {
124          statusLabel.setForeground(error ? new Color(170, 35, 35) : AppTheme.INK);
125          statusLabel.setText(text);
126      }
127
128      private void showDate() {
129          try { anchor=LocalDate.parse(anchorField.getText().trim()); refresh(); }
```

Time Matters Java program

src/demo/SchedulerFrame.java continued

```
130         catch (RuntimeException ex) { message("Enter a valid view date as YYYY-MM-DD.",true); }
131     }
132
133     private void move(int direction) {
134         String period=(String)periodBox.getSelectedItem();
135         anchor=switch(period) {
136             case "Week" -> anchor.plusWeeks(direction);
137             case "Month" -> anchor.plusMonths(direction);
138             default -> anchor.plusDays(direction);
139         };
140         refresh();
141     }
142
143     private void refresh() {
144         anchorField.setText(anchor.toString());
145         visible=book==null ? List.of() : book.view(anchor,(String)periodBox.getSelectedItem());
146         model.setRowCount(0);
147         for (Occurrence occurrence : visible) model.addRow(occurrence.toTableRow());
148         exportButton.setEnabled(!visible.isEmpty());
149         updateButton.setEnabled(false);
150         deleteButton.setEnabled(false);
151     }
152
153     private void exportView() {
154         List<Occurrence> rows = List.copyOf(visible);
155         if (rows.isEmpty()) return;
156         ZoneId zone = ZoneId.systemDefault();
157         JFileChooser chooser = new JFileChooser();
158         chooser.setDialogTitle("Export current view (" + zone + ")");
159         chooser.setFileFilter(new javax.swing.filechooser.FileNameExtensionFilter("iCalendar (*.ics)", "ics"));
160         chooser.setSelectedFile(new java.io.File("appointments-" + anchor + ".ics"));
161         if (chooser.showSaveDialog(this) != JFileChooser.APPROVE_OPTION) return;
162         Path target = chooser.getSelectedFile().toPath();
163         if (!target.getFileName().toString().toLowerCase(Locale.ROOT).endsWith(".ics")) {
164             target = target.resolveSibling(target.getFileName() + ".ics");
165         }
166         boolean exists = Files.exists(target);
167         if (exists && JOptionPane.showConfirmDialog(this, "Replace " + target.getFileName() + "?",
168                 "Replace calendar file", JOptionPane.YES_NO_OPTION) != JOptionPane.YES_OPTION) return;
169         try {
170             CalendarExport.write(target, rows, zone, exists);
171             message("Exported " + rows.size() + " appointment(s) to " + target.getFileName() + ".", false);
172         } catch (IOException | DateTimeException | IllegalArgumentException ex) {
```

Time Matters Java program

src/demo/SchedulerFrame.java continued

```
173             message("Could not export: " + ex.getMessage(), true);
174         }
175     }
176 
177     private void select() {
178         int row=table.getSelectedRow();
179         boolean selected=row>=0 && row<visible.size();
180         updateButton.setEnabled(selected && book!=null);
181         deleteButton.setEnabled(selected && book!=null);
182         if (!selected) return;
183         Appointment appointment = visible.get(row).appointment();
184         Object[] values=appointment.toTableRow();
185         JTextField[] fields={titleField,dateField,startField,endField,locationField};
186         for(int i=0;i<fields.length;i++) fields[i].setText(values[i].toString());
187         repeatBox.setSelectedItem(appointment.getRepeat());
188         untilField.setText(appointment.getRepeat() == Appointment.Repeat.NONE
189                 ? "" : appointment.getRepeatUntil().toString());
190         reminderField.setValue(appointment.getReminderMinutes());
191         if (appointment.getRepeat() != Appointment.Repeat.NONE) {
192             message("Editing or deleting this row affects the whole series.", false);
193         }
194     }
195 
196     private void clear() {
197         table.clearSelection();
198         titleField.setText(""); locationField.setText("");
199         dateField.setText(anchor.toString()); startField.setText("09:00"); endField.setText("10:00");
200         repeatBox.setSelectedItem(Appointment.Repeat.NONE);
201         untilField.setText("");
202         reminderField.setValue(0);
203     }
204 
205     private void save(boolean editing) {
206         if(book==null) return;
207         int row=table.getSelectedRow();
208         if(editing && row<0) { message("Select an appointment first.",true); return; }
209         try {
210             Appointment input=Appointment.fromInput(titleField.getText(),dateField.getText(),
211                     startField.getText(),endField.getText(),locationField.getText());
212             Appointment.Repeat repeat = (Appointment.Repeat) repeatBox.getSelectedItem();
213             LocalDate until = repeat == Appointment.Repeat.NONE ? null : LocalDate.parse(untilField.getText().trim());
214             reminderField.commitEdit();
215             Appointment a = new Appointment(input.getTitle(), input.getDate(), input.getStart(),
```

Time Matters Java program

src/demo/SchedulerFrame.java continued

```
                    input.getEnd(), input.getLocation(), repeat, until, (Integer) reminderField.getValue());
            if(editing) book.update(book.getAll().indexOf(visible.get(row).appointment()),a);
            else book.add(a);
            anchor=a.getDate(); refresh(); clear();
            message((editing ? "Updated" : "Added") + ": " + a.getTitle()+". Saved. Total: "+book.getAll().size(),false);
        } catch (DateTimeException ex) {
            message("Enter a valid repeat end date as YYYY-MM-DD.", true);
        } catch (java.text.ParseException ex) {
            message("Enter a reminder from 0 to 1440 minutes.", true);
        } catch (IllegalArgumentException | IOException ex) { message("Not saved: "+ex.getMessage(),true); }
    }

    private void deleteSelected() {
        int row=table.getSelectedRow();
        if(book==null || row<0) return;
        Appointment a=visible.get(row).appointment();
        String item = a.getRepeat() == Appointment.Repeat.NONE ? a.getTitle() : "the entire series " + a.getTitle();
        if(JOptionPane.showConfirmDialog(this,"Delete " + item + "?", "Delete appointment",
                JOptionPane.YES_NO_OPTION)!=JOptionPane.YES_OPTION) return;
        try {
            book.delete(book.getAll().indexOf(a)); refresh(); clear();
            message("Deleted: " + a.getTitle() + ". Saved.",false);
        } catch(IOException ex) { message("Not deleted: "+ex.getMessage(),true); }
    }

    @SuppressWarnings({"unchecked", "rawtypes"})
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    private void initComponents() {
        topPanel = new javax.swing.JPanel();
        topPanel.setLayout(new java.awt.BorderLayout(8, 8));
        navigationPanel = new javax.swing.JPanel();
        navigationPanel.setLayout(new java.awt.FlowLayout(java.awt.FlowLayout.LEFT));
        viewLabel = new javax.swing.JLabel();
        viewLabel.setText("View");
        navigationPanel.add(viewLabel);
        periodBox = new javax.swing.JComboBox();
        navigationPanel.add(periodBox);
        anchorLabel = new javax.swing.JLabel();
        anchorLabel.setText("Date (YYYY-MM-DD)");
        navigationPanel.add(anchorLabel);
        anchorField = new javax.swing.JTextField();
        anchorField.setText("");
        navigationPanel.add(anchorField);
```

Time Matters Java program

src/demo/SchedulerFrame.java continued

```
259         showButton = new javax.swing.JButton();
260         showButton.setText("Show");
261         navigationPanel.add(showButton);
262         previousButton = new javax.swing.JButton();
263         previousButton.setText("Previous");
264         navigationPanel.add(previousButton);
265         todayButton = new javax.swing.JButton();
266         todayButton.setText("Today");
267         navigationPanel.add(todayButton);
268         nextButton = new javax.swing.JButton();
269         nextButton.setText("Next");
270         navigationPanel.add(nextButton);
271         topPanel.add(navigationPanel, java.awt.BorderLayout.NORTH);
272         fieldsPanel = new javax.swing.JPanel();
273         fieldsPanel.setLayout(new java.awt.GridLayout(2, 5, 10, 6));
274         titleLabel = new javax.swing.JLabel();
275         titleLabel.setText("Title (required)");
276         fieldsPanel.add(titleLabel);
277         dateLabel = new javax.swing.JLabel();
278         dateLabel.setText("Date (YYYY-MM-DD)");
279         fieldsPanel.add(dateLabel);
280         startLabel = new javax.swing.JLabel();
281         startLabel.setText("Start (HH:mm)");
282         fieldsPanel.add(startLabel);
283         endLabel = new javax.swing.JLabel();
284         endLabel.setText("End (HH:mm)");
285         fieldsPanel.add(endLabel);
286         locationLabel = new javax.swing.JLabel();
287         locationLabel.setText("Location (optional)");
288         fieldsPanel.add(locationLabel);
289         titleField = new javax.swing.JTextField();
290         titleField.setText("");
291         fieldsPanel.add(titleField);
292         dateField = new javax.swing.JTextField();
293         dateField.setText("");
294         fieldsPanel.add(dateField);
295         startField = new javax.swing.JTextField();
296         startField.setText("");
297         fieldsPanel.add(startField);
298         endField = new javax.swing.JTextField();
299         endField.setText("");
300         fieldsPanel.add(endField);
301         locationField = new javax.swing.JTextField();
```

Time Matters Java program

src/demo/SchedulerFrame.java continued

```
302         locationField.setText("");
303         fieldsPanel.add(locationField);
304         topPanel.add(fieldsPanel, java.awt.BorderLayout.CENTER);
305         actionsPanel = new javax.swing.JPanel();
306         actionsPanel.setLayout(new java.awt.FlowLayout(java.awt.FlowLayout.LEFT));
307         addButton = new javax.swing.JButton();
308         addButton.setText("Add appointment");
309         actionsPanel.add(addButton);
310         updateButton = new javax.swing.JButton();
311         updateButton.setText("Save changes");
312         actionsPanel.add(updateButton);
313         deleteButton = new javax.swing.JButton();
314         deleteButton.setText("Delete selected");
315         actionsPanel.add(deleteButton);
316         clearButton = new javax.swing.JButton();
317         clearButton.setText("Clear selection");
318         actionsPanel.add(clearButton);
319         exportButton = new javax.swing.JButton();
320         exportButton.setText("Export view (.ics)");
321         actionsPanel.add(exportButton);
322         topPanel.add(actionsPanel, java.awt.BorderLayout.SOUTH);
323         getContentPane().add(topPanel, java.awt.BorderLayout.NORTH);
324         scrollPane = new javax.swing.JScrollPane();
325         table = new javax.swing.JTable();
326         scrollPane.setViewportView(table);
327         getContentPane().add(scrollPane, java.awt.BorderLayout.CENTER);
328         statusLabel = new javax.swing.JLabel();
329         statusLabel.setText("Select an appointment to edit or delete.");
330         getContentPane().add(statusLabel, java.awt.BorderLayout.SOUTH);
331         setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
332         setTitle("Appointment Scheduler - Day 5");
333         pack();
334     }// </editor-fold>//GEN-END:initComponents
335 
336     // Variables declaration - do not modify//GEN-BEGIN:variables
337     private javax.swing.JPanel topPanel;
338     private javax.swing.JPanel navigationPanel;
339     private javax.swing.JLabel viewLabel;
340     private javax.swing.JComboBox periodBox;
341     private javax.swing.JLabel anchorLabel;
342     private javax.swing.JTextField anchorField;
343     private javax.swing.JButton showButton;
344     private javax.swing.JButton previousButton;
```

Time Matters Java program

src/demo/SchedulerFrame.java continued

```
345      private javax.swing.JButton todayButton;
346      private javax.swing.JButton nextButton;
347      private javax.swing.JPanel fieldsPanel;
348      private javax.swing.JLabel titleLabel;
349      private javax.swing.JLabel dateLabel;
350      private javax.swing.JLabel startLabel;
351      private javax.swing.JLabel endLabel;
352      private javax.swing.JLabel locationLabel;
353      private javax.swing.JTextField titleField;
354      private javax.swing.JTextField dateField;
355      private javax.swing.JTextField startField;
356      private javax.swing.JTextField endField;
357      private javax.swing.JTextField locationField;
358      private javax.swing.JPanel actionsPanel;
359      private javax.swing.JButton addButton;
360      private javax.swing.JButton updateButton;
361      private javax.swing.JButton deleteButton;
362      private javax.swing.JButton clearButton;
363      private javax.swing.JButton exportButton;
364      private javax.swing.JScrollPane scrollPane;
365      private javax.swing.JTable table;
366      private javax.swing.JLabel statusLabel;
367      // End of variables declaration//GEN-END:variables
368  }
```